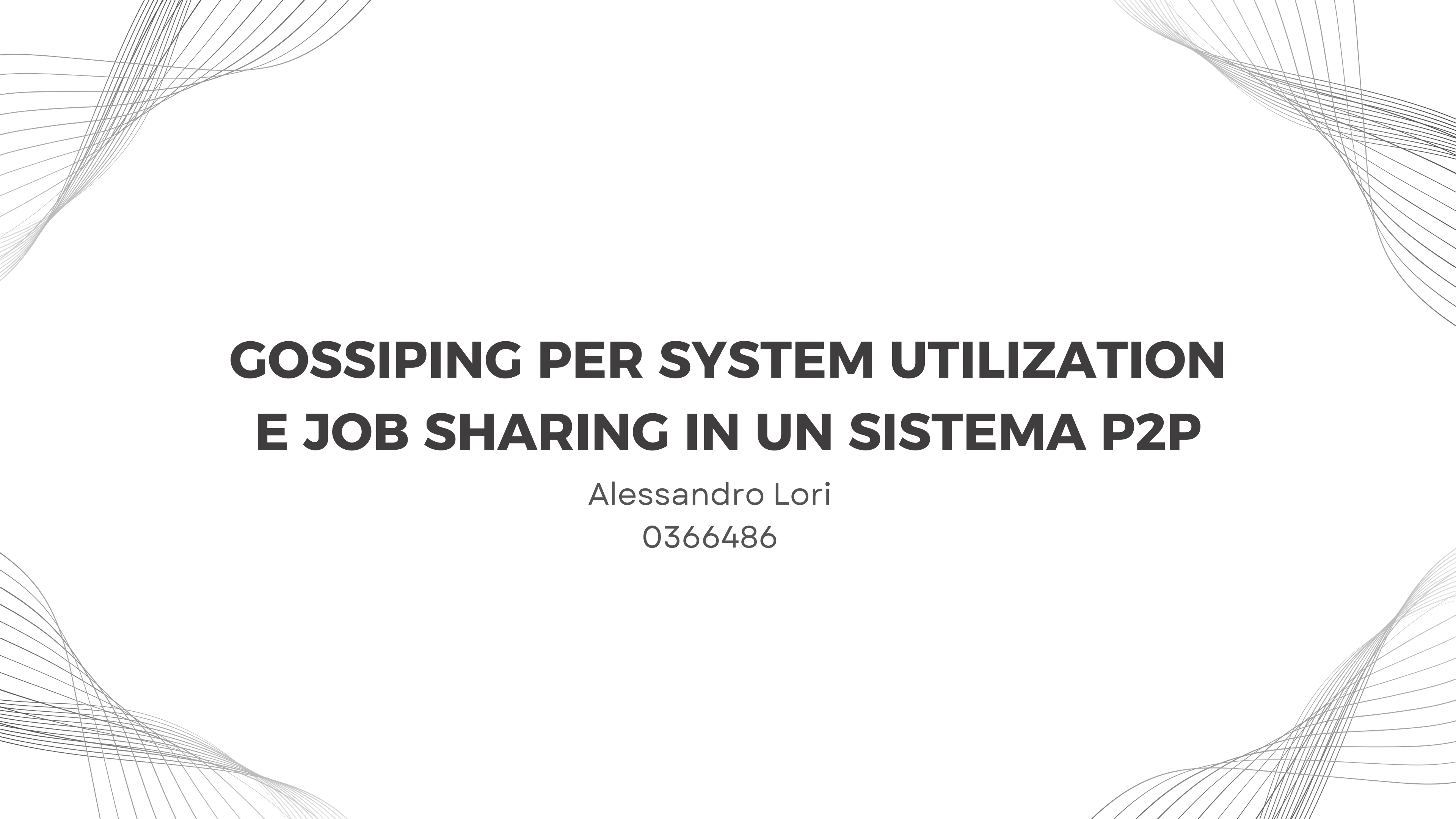
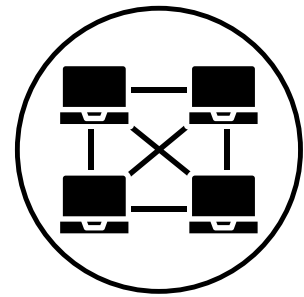




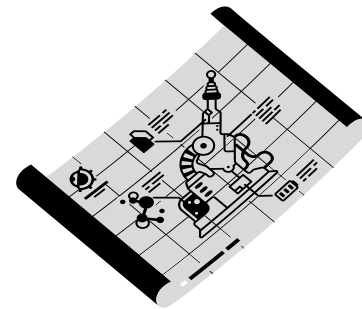
GOSSIPING PER SYSTEM UTILIZATION E JOB SHARING IN UN SISTEMA P2P

Alessandro Lori
0366486

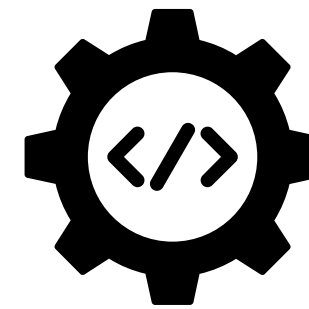
Struttura del Progetto



Protocollo



Architettura



Implementazione



Conclusioni

Protocollo Gossip

- Meccanismo per la comunicazione distribuita, permette:
 - Controllo decentralizzato.
 - Basso carico di banda di rete.
 - Scalabilità.
 - Semplicità di gestione.
- Ogni nodo della rete P2P periodicamente scambia conoscenze con un sottogruppo di nodi tra quelli appartenenti alla sua Membership List.
- Ogni novità ottenuta da un nodo viene propagata dal nodo stesso rendendo gli aggiornamenti convergenti su tutta la rete in tempo finito.

Protocollo Gossip - Scambio Anti-Entropy

Lo scambio di tipo Anti-Entropy garantisce consistenza delle informazioni nella rete diminuendo la situazione di disordine tra i nodi.
3 strategie diverse attuabili:

- Push: Il nodo invia informazioni al destinatario che confronta ed eventualmente aggiorna.
- Pull: Il nodo richiede informazioni al destinatario per poi confrontare ed eventualmente aggiornare.
- Push-Pull: Il nodo invia e richiede informazioni al destinatario, entrambi confrontano ed eventualmente aggiornano.

Architettura di sistema

L'architettura mira a simulare situazioni reali in una rete P2P nella quale i nodi:

- Scambiano tra loro informazioni sull'utilizzazione delle componenti proprie e degli altri nodi.
- Generano job, da scambiarsi, di carico diverso CPU, MEM, GPU.
- Effettuano operazioni di PING per monitorare lo stato di vita degli altri nodi.
- Hanno una lista personale di Friends&Affinity per decidere a chi affidare il job generato.
- Sono soggetti a Fault e Leave volontarie.

Architettura di sistema - Tecnologie

Le tecnologie usate nello sviluppo:

- Golang: linguaggio di programmazione sviluppato da Google.
- gRPC: Framework RPC open-source, ci permette di eseguire chiamate remote come fossero locali garantendo comunicazioni bidirezionali e basse latenze.
- Piggyback: piccoli payload allegati ai metadata gRPC, nell'header del messaggio HTTP/2.
- Docker e Docker Compose: piattaforma per eseguire servizi in container e gestire l'interazione tra essi.

Architettura di sistema - Patterns

I patterns che accompagnano le tecnologie nello sviluppo:

- ExchangeAvail: scambio tra nodi di messaggi Anti-Entropy Push-Pull.
- SWIM: scambio tra nodi di PING e PINGREQ per monitorare lo stato di vita.
- Scheduling: scambio tra nodi di messaggi PROBE, CANCEL, COMMIT per schedulare il job generato:
 - PROBE: sonda alcuni vicini per avere disponibilità.
 - CANCEL: annulla la richiesta di disponibilità di un vicino.
 - COMMIT: invia il job ad un vicino disponibile .

Implementazione

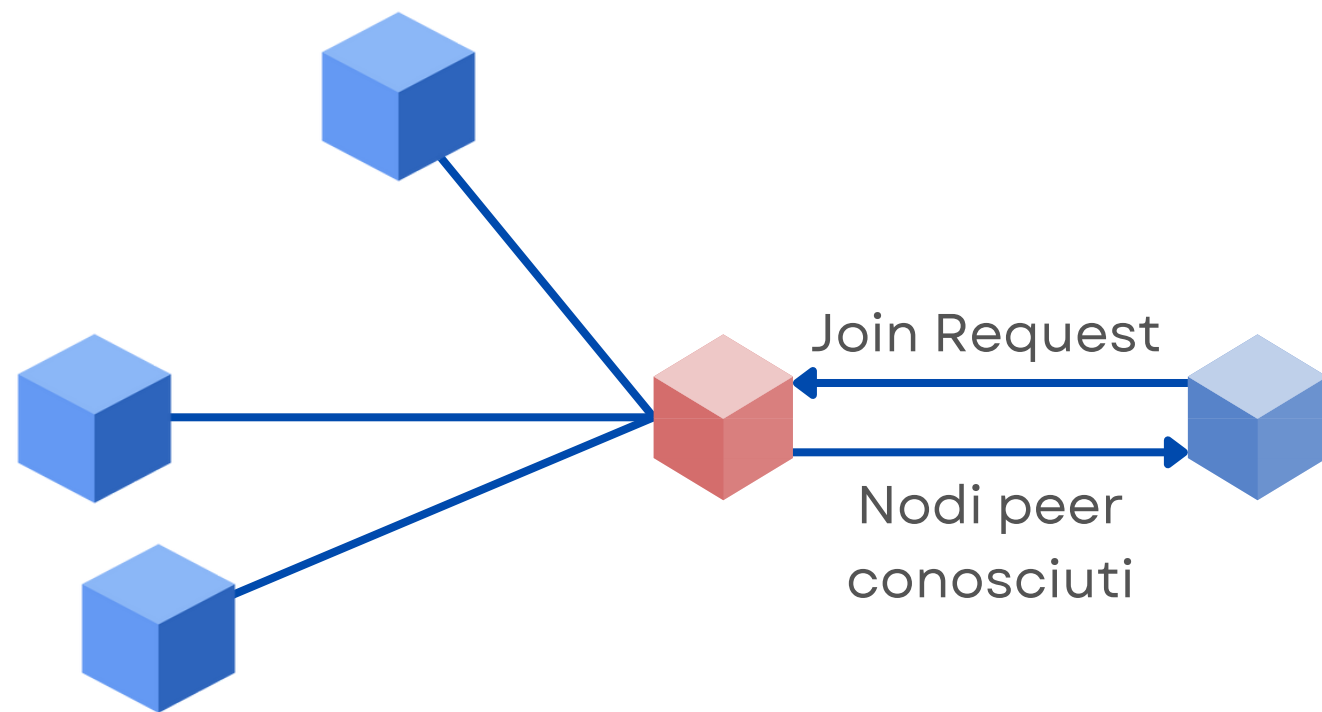
L'implementazione dell'architettura presenta la simulazione automatica del sistema in funzione di molteplici bucket di probabilità.

La generazione di un nodo ad inizio simulazione è caratterizzata da probabilità che ne definiscono la natura operativa nel sistema, descrivono:

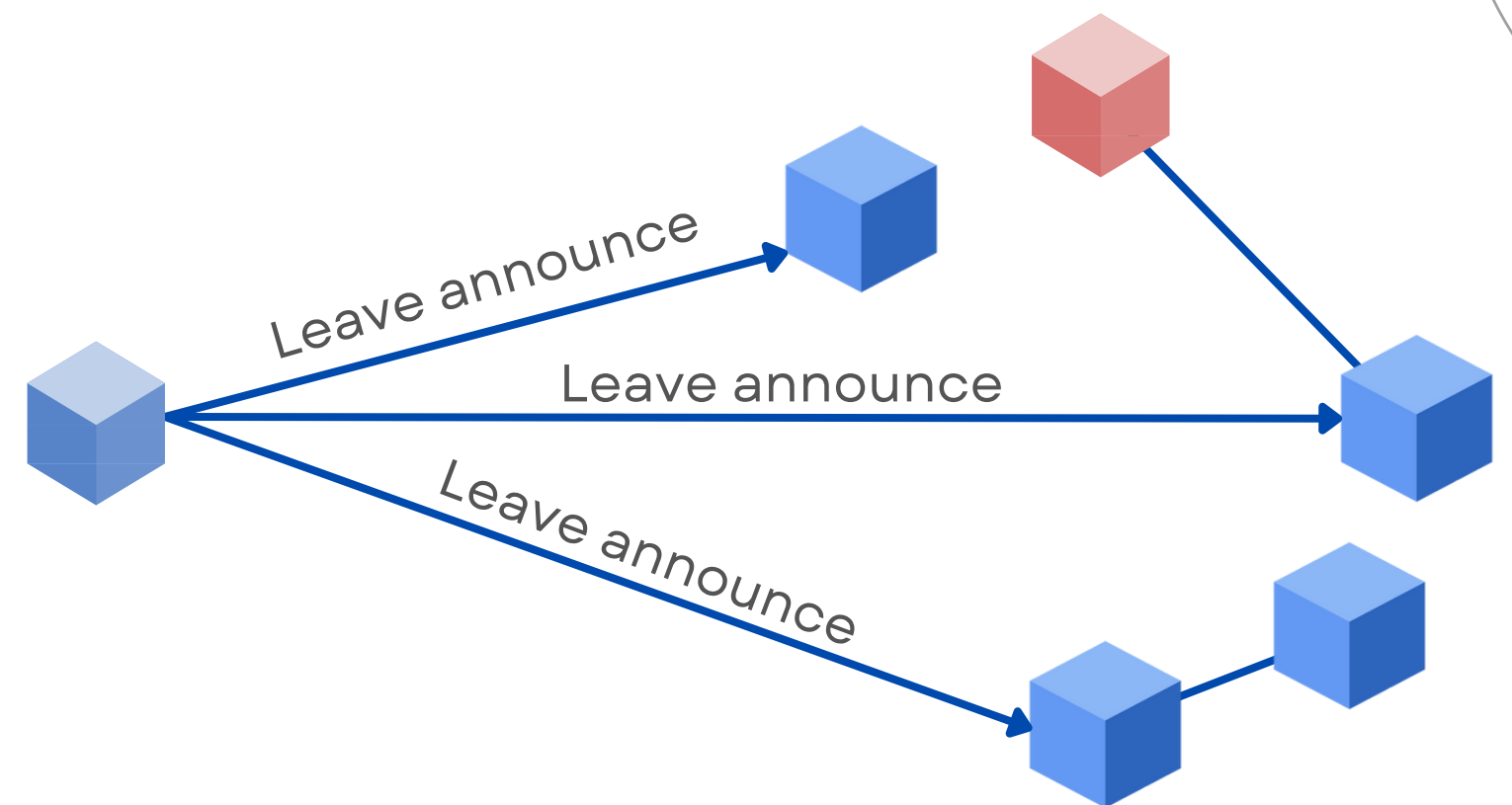
- Quantità, frequenza e tipologia di job generati
- Frequenza di Fault e Leave volontarie
- Carico di sistema iniziale

Implementazione - Join&Leave

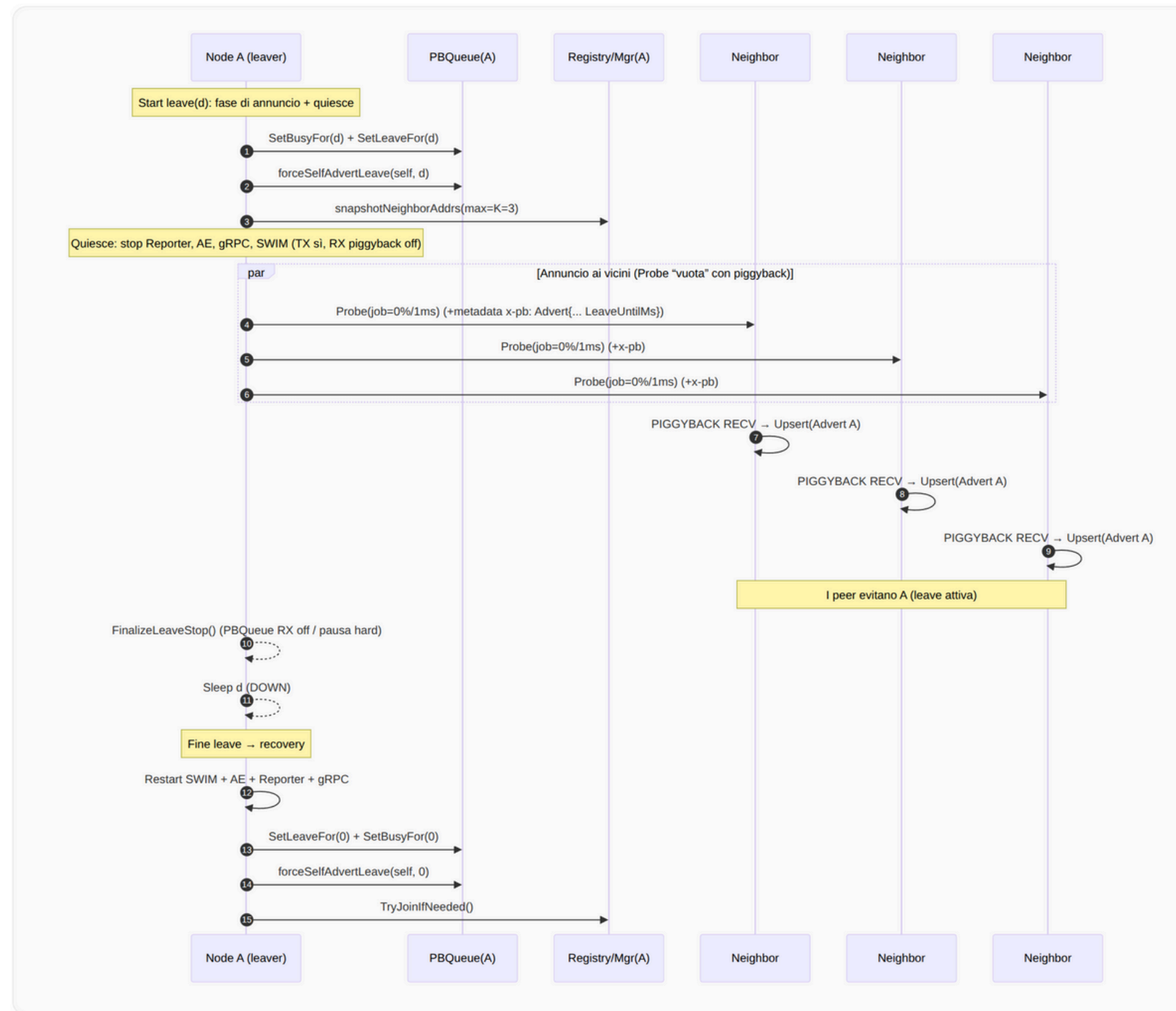
Join:



Leave:

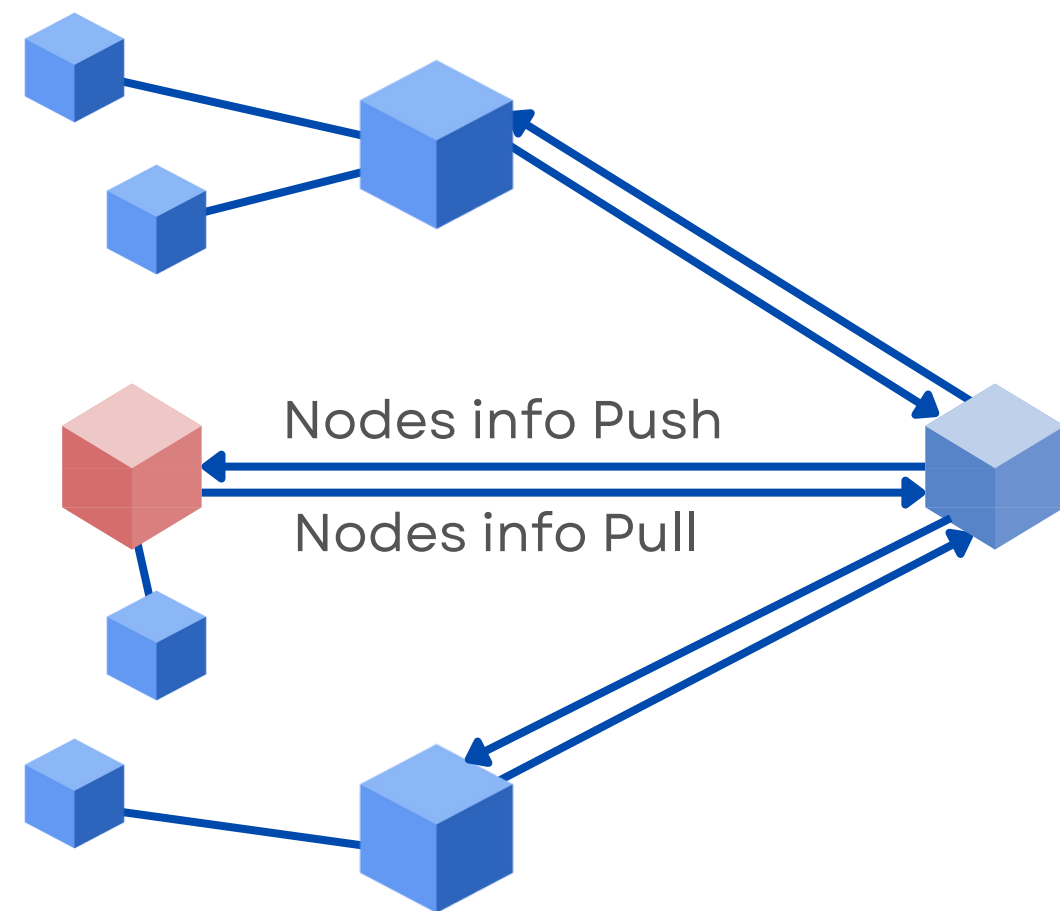


Implementazione - Leave Sequence



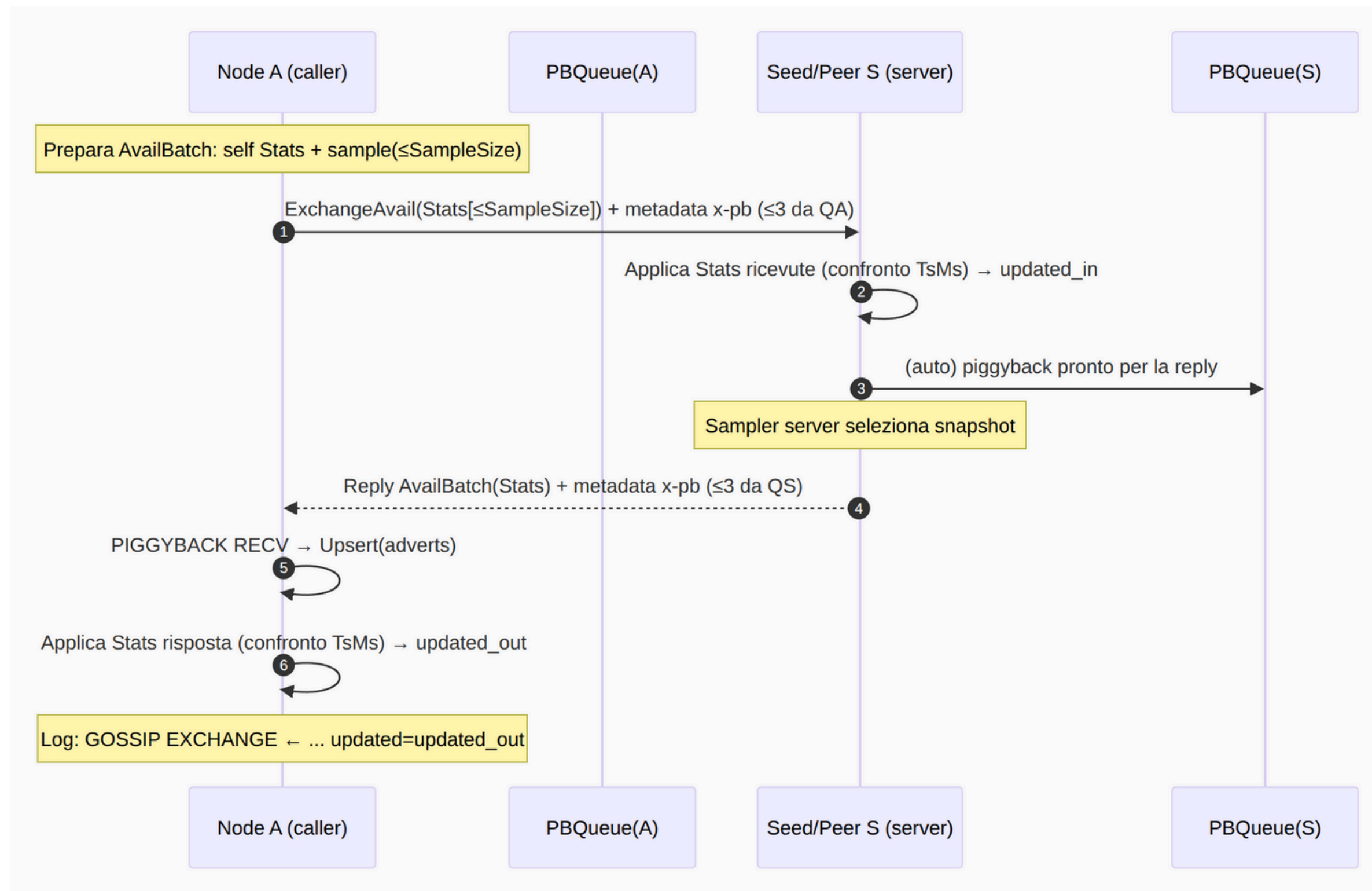
Implementazione - ExchangeAvail

Gossip esplicito, i nodi scambiano le proprie conoscenze, di massimo 8 nodi conosciuti, per poi aggiornare localmente.



```
AVAIL UPDATE → node-066b61c73851 cpu=10.0% mem=12.0% gpu=4.0% ts=300254
AVAIL UPDATE → node-4761e33991e0 cpu=10.0% mem=12.0% gpu=-1.0% ts=330111
AVAIL UPDATE → node-555349f69014 cpu=18.7% mem=92.1% gpu=18.8% ts=274978
AVAIL UPDATE → node-e59afa873681 cpu=8.0% mem=10.0% gpu=3.0% ts=262433
GOSSIP EXCHANGE ← 127.0.0.1:9004 updated=4
```

Implementazione - Exchange Sequence



Implementazione - Job Generation

Ogni job è caratterizzato da:

- Carico su CPU, MEM e GPU.
- Durata media stimata.

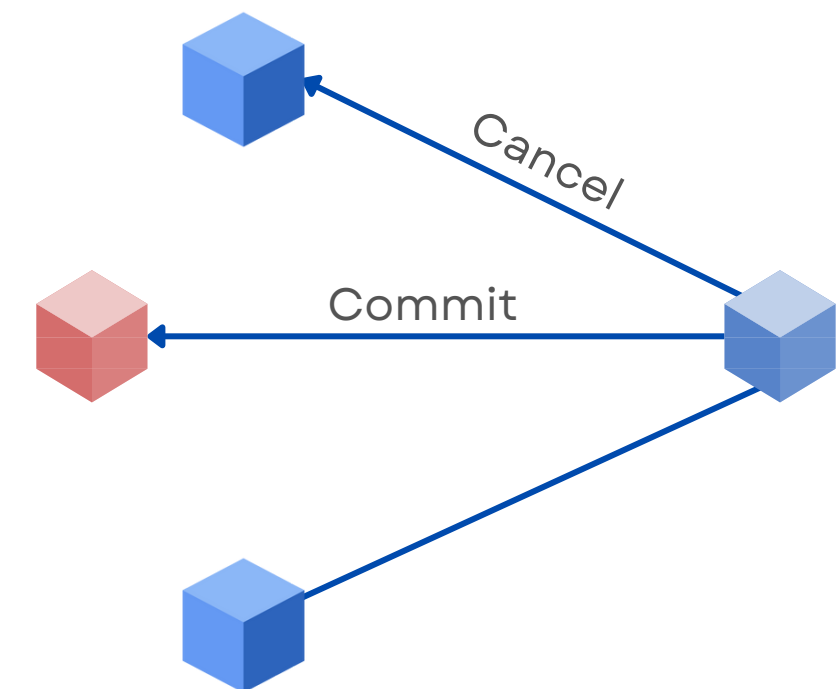
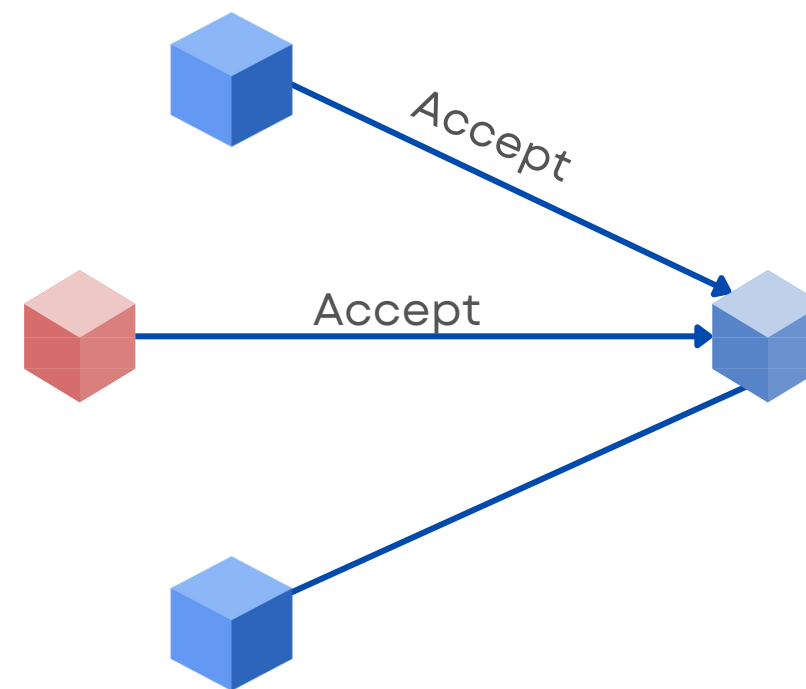
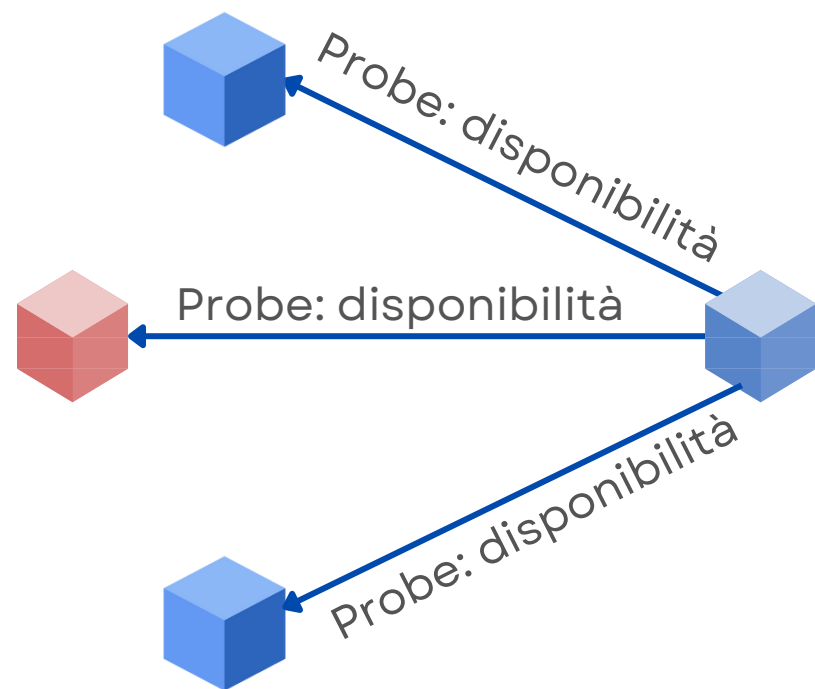
Ogni nodo ha una propensione a generare tipi di job diversi:

- CPU_HEAVY: carico maggiormente distribuito sulla CPU.
- MEM_HEAVY: carico maggiormente distribuito sulla MEM.
- GPU_HEAVY: carico maggiormente distribuito sulla GPU.
- GENERAL: carico equamente distribuito tra le parti.

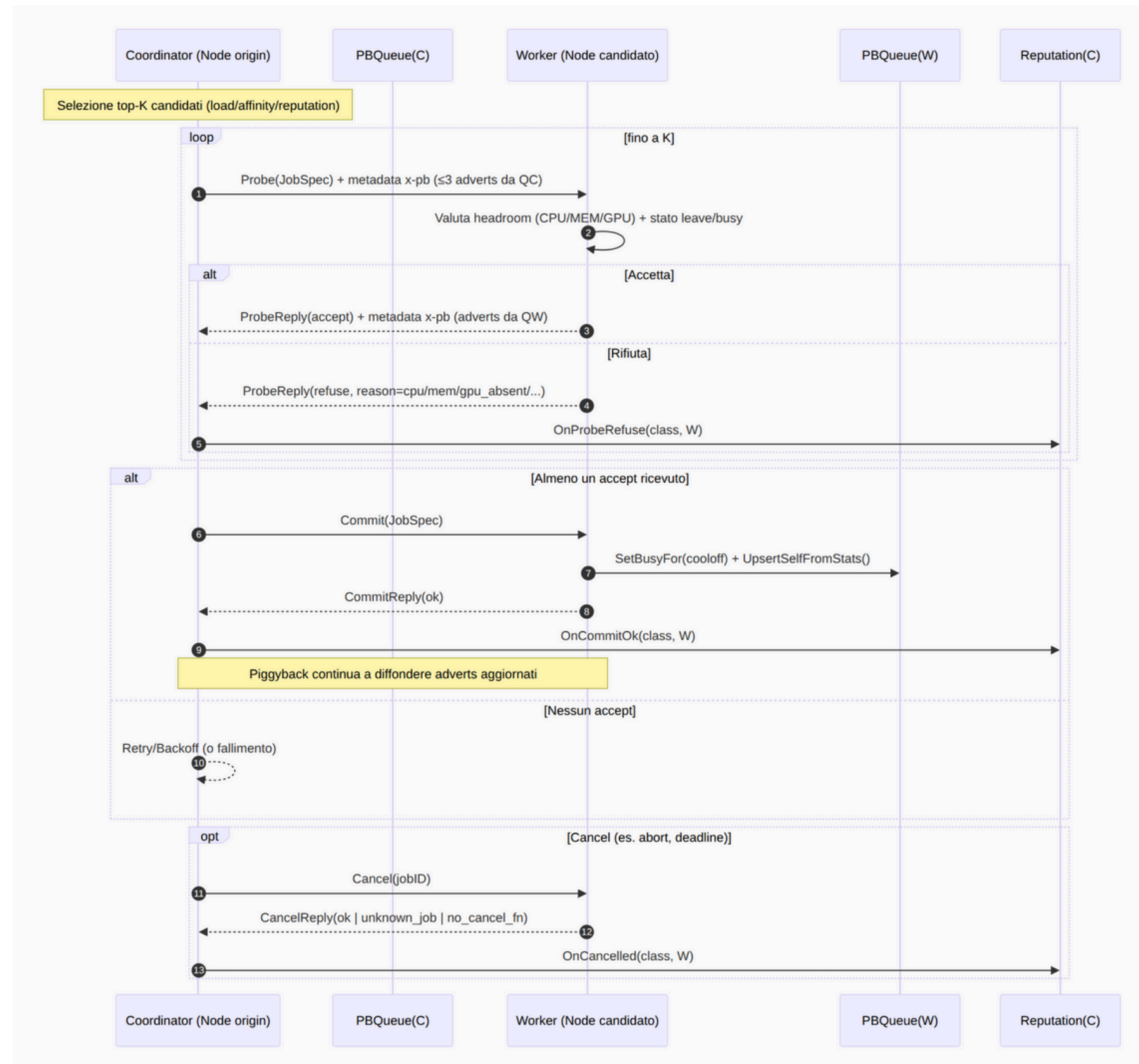
La generazione delle propensioni e dei singoli job è in funzione di bucket di probabilità che permettono anche la presenza di job ibridi con alto carico su più componenti generando ad esempio CPU-MEM_HEAVY jobs

Implementazione - Job Scheduling

Il nodo sonda la disponibilità di altri 3 nodi facenti parte della sua Membership List per poi affidare il job generato.



Implementazione - Scheduling Sequence



Implementazione - Friends&Affinity

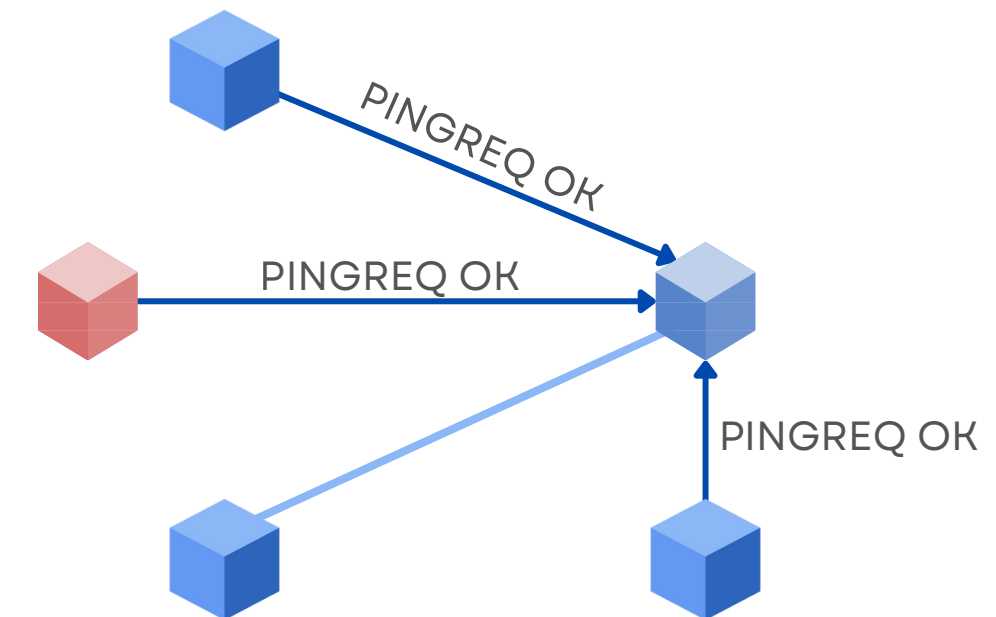
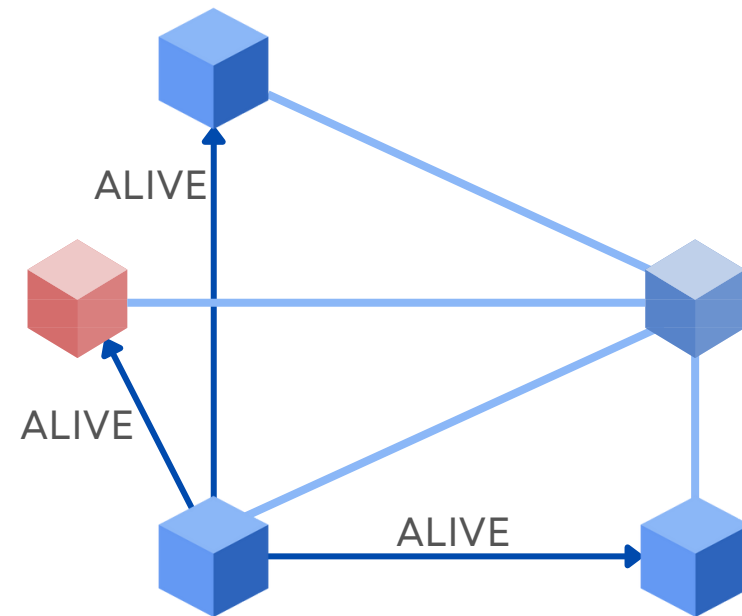
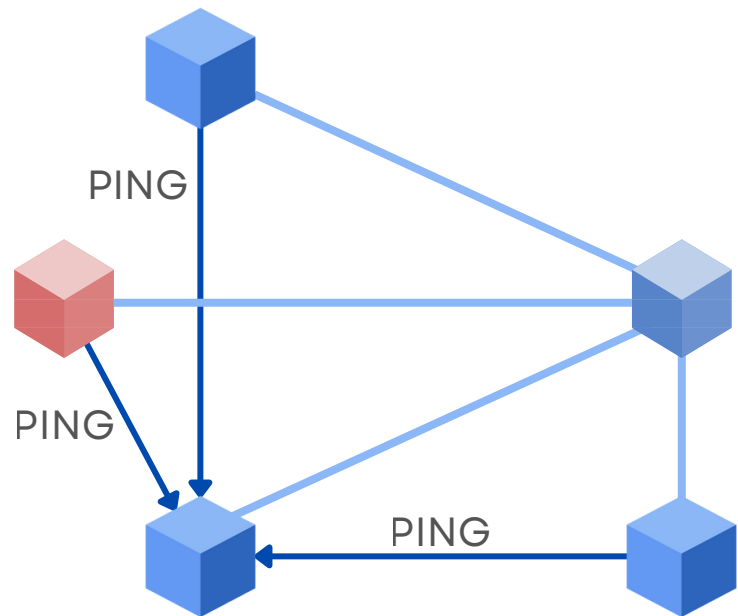
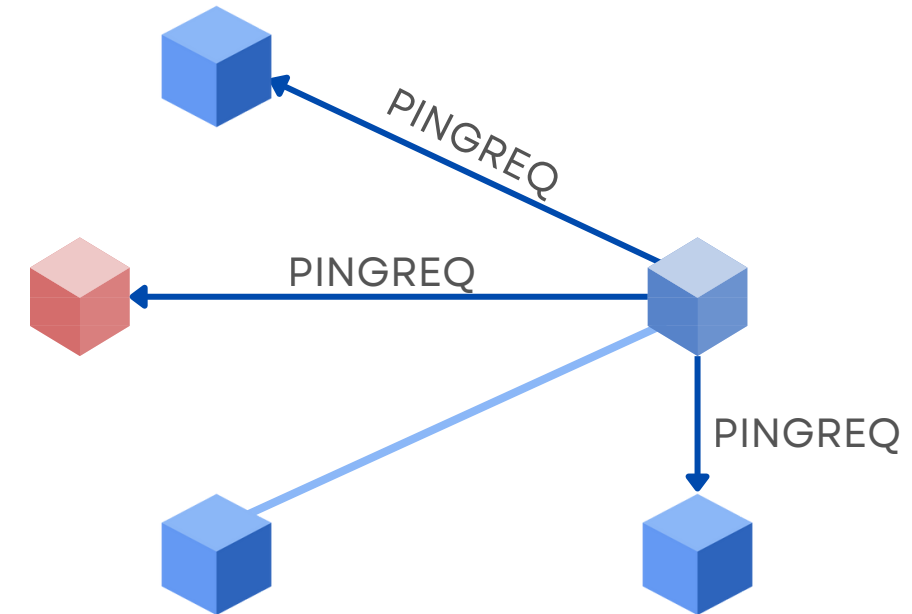
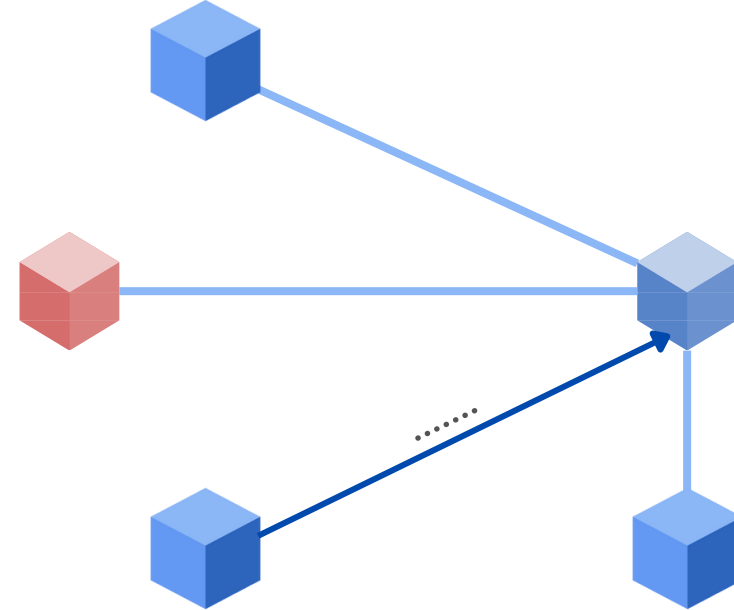
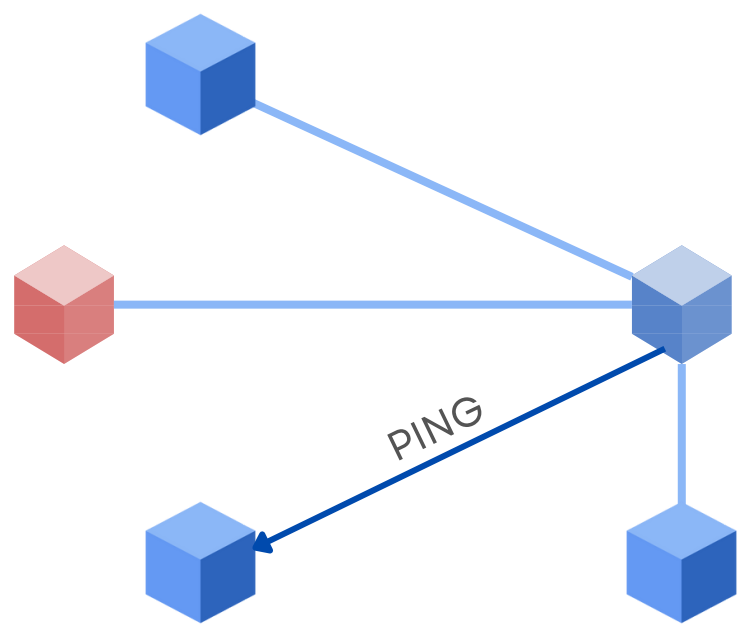
Ogni nodo ha una lista locale tramite la quale riconosce chi è più adatto ad un determinato job, rendendolo suo amico.

Per evitare isolazionismo ed un effetto gregge ogni nodo ha una tabella di affinità e reputazione composta da score per ogni nodo che:

- Decade a zero nel tempo tramite normalizzazione.
- Decrementa per ogni job rifiutato.
- Incrementa per ogni job accettato.
- Incrementa per ogni job eseguito correttamente.

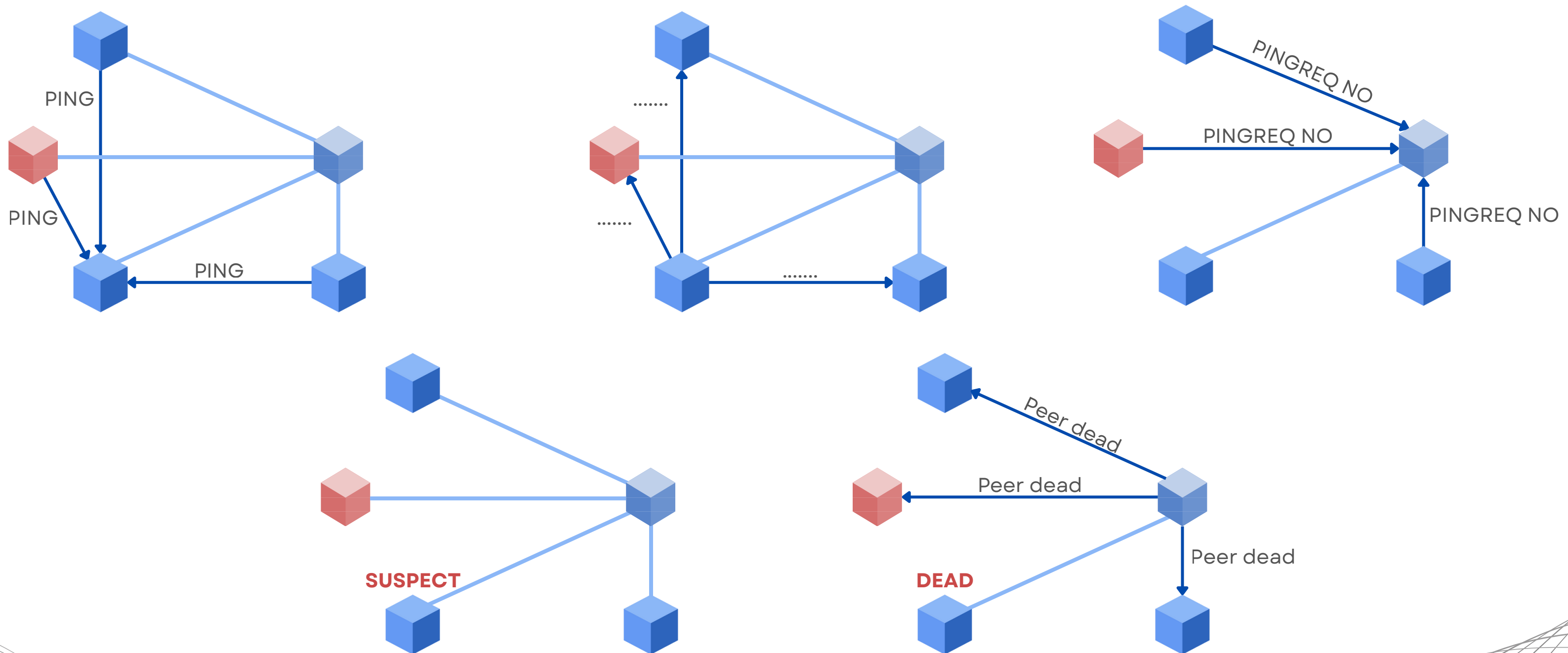
Un meccanismo di Cooloff per ogni azione intrapresa tra nodi permette di non eseguire stesse richieste e di lasciare disponibilità anche ad altri nodi.

Implementazione - SWIM Ping di Stato



Implementazione - Fault

Durante lo SWIM se un nodo ha subito un fallimento:



Implementazione - Piggyback

Ogni messaggio inviato dai nodi contiene ulteriori informazioni nei metadata permettendo un Gossip indiretto che contribuisce alla diffusione delle informazioni. All'interno dei metadata i nodi scambiano:

- Informazioni di utilizzazione di sistema.
- Flag per la presenza della GPU.
- Campo busy che indica quanto tempo il nodo sarà occupato.

Nel caso della Leave il piggyback conterrà oltre alle altre informazioni anche l'avviso di Leave del nodo.

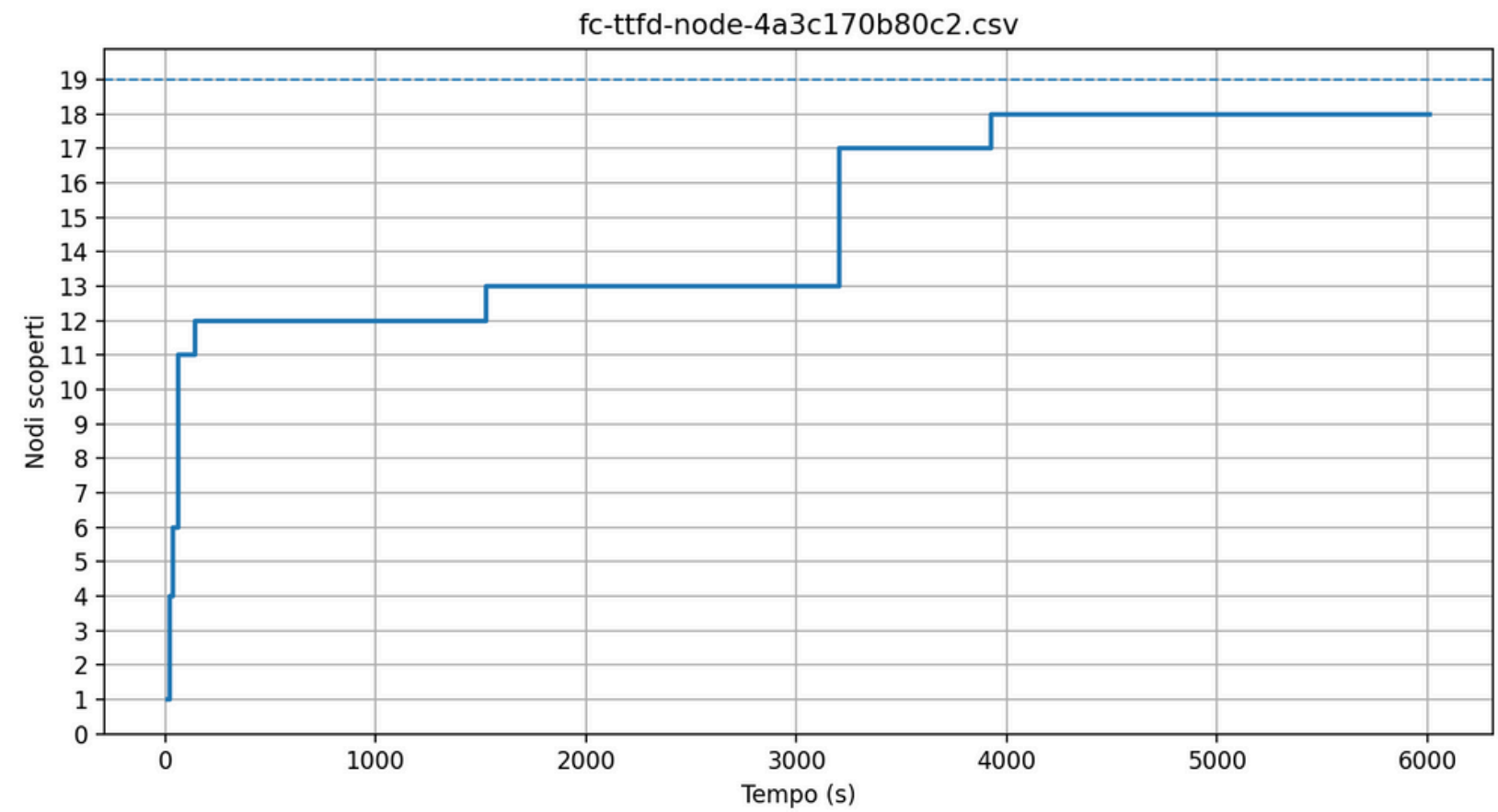
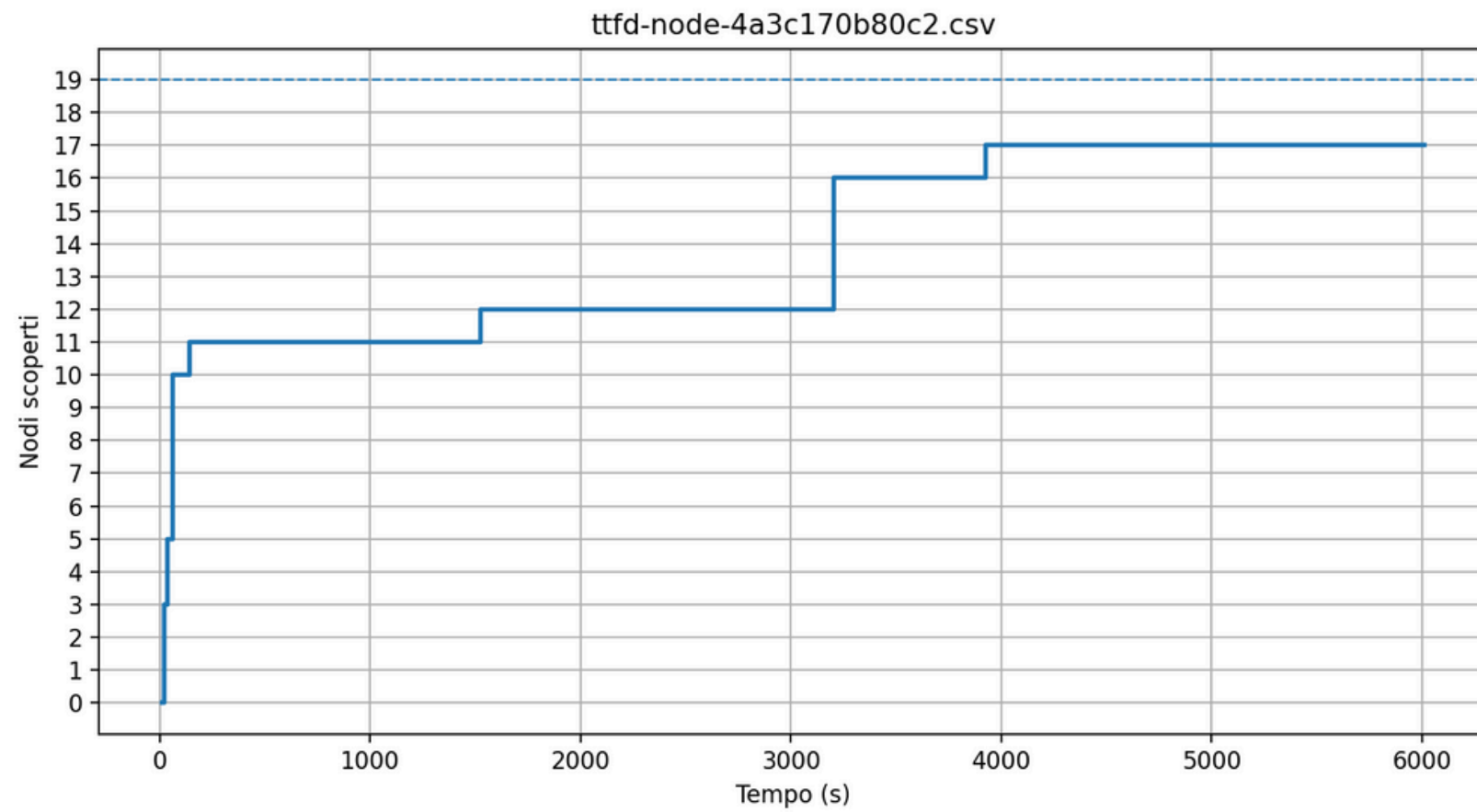
Implementazione - Scoperta della Rete

Ogni nodo ha 2 modalità di scoperta della rete:

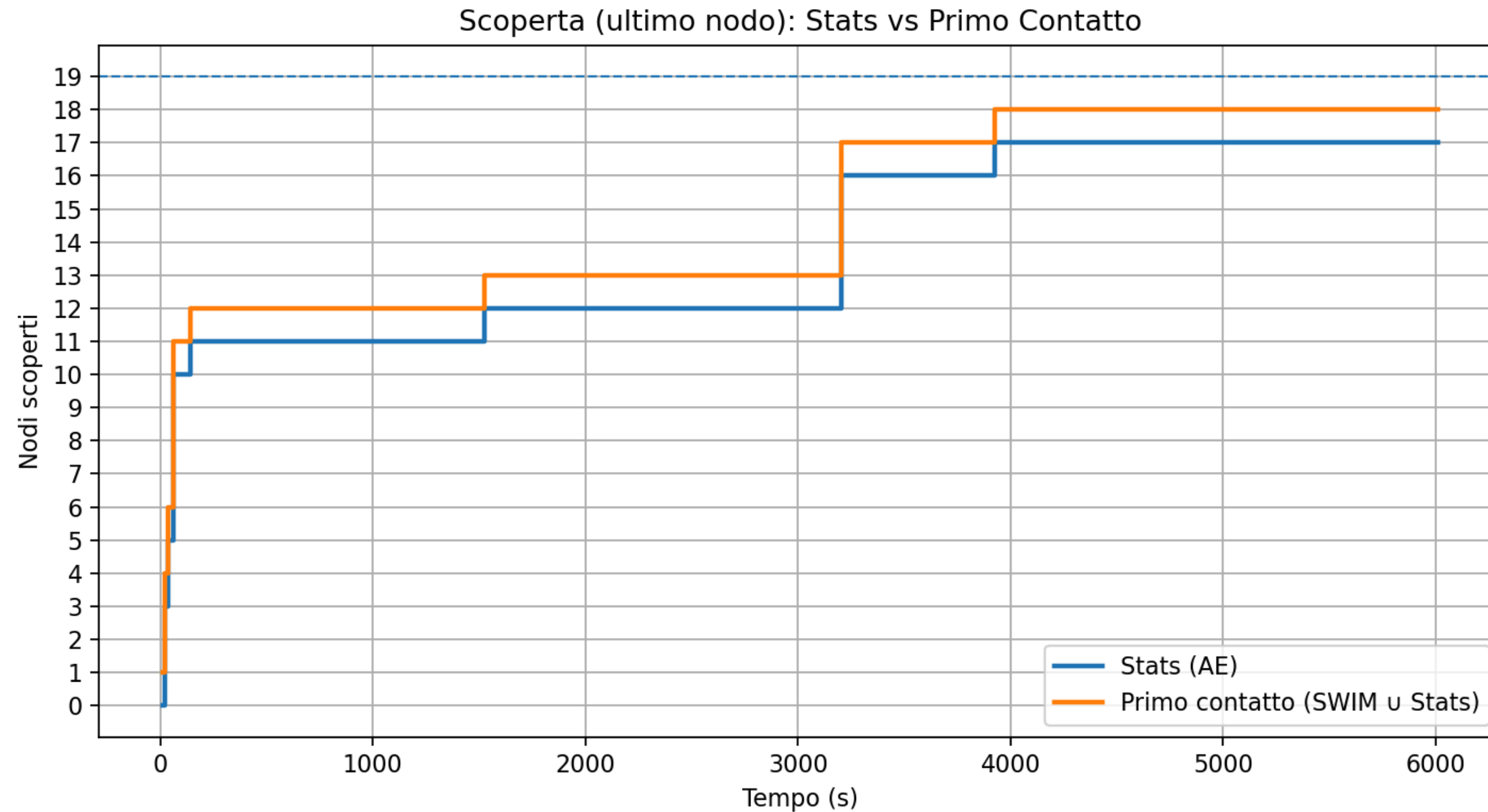
- TTFD - Time To First Data: il nodo scopre l'esistenza di un altro nodo ottenendone le relative stats di utilizzazione.
- FC-TTFD - First Contract-TTFD: il nodo scopre l'esistenza di un altro nodo senza avere ulteriori informazioni oltre alla sua presenza nella rete.

Con simulazione già iniziata l'ultimo nodo entrato incontrerà maggiore semplicità nello scoprire la rete con modalità FC-TTFD data l'elevata presenza di Gossip implicito.

Implementazione - Scoperta della Rete



Implementazione - Scoperta della Rete



Conclusioni

L'architettura implementata simula con risultato atteso un sistema P2P per System Utilization e Job Sharing attraverso l'uso di gRPC per la comunicazione tra nodi e del protocollo Gossiping per la diffusione delle informazioni nella rete.

Grazie all'elevata varianza data dai bucket di probabilità l'architettura riesce a riprodurre un contesto realistico come quello affrontato nelle render farm o mining pool.



GRAZIE PER L'ATTENZIONE